

AUDIT — KZTEK Image Tools (`tool/`)

- Ngày: 2026-07-26
- Phạm vi: toàn bộ package `tool/` (89 file `.py`, 44.608 dòng) + cấu hình/đóng gói liên quan
- Loại: Audit → đã thực hiện phần lớn đề xuất (xem §0b)
- Priority: P1
- Nhánh: `update` — commit gần nhất `9754176` (2026-07-24)

0. Tóm tắt điều hành

Codebase đã được tổ chức lại tốt (`core/` - `features/` - `shared/` - `utils/`, chia theo domain: `annotation / training / detection / dataset / collection / analysis`) và **không có lỗi cú pháp nào** — `compileall` sạch 89/89 file. Chất lượng nền tốt hơn mức trung bình của tool nội bộ: 0 `bare except`, 0 mutable default arg, 0 `os.system`, 0 `verify=False`.

Tuy nhiên có **4 vấn đề P1** cần xử lý sớm, trong đó **1 vấn đề bảo mật đã bị đẩy lên GitHub remote công khai**.

#	Vấn đề	Mức	Effort ước tính
---	-----	----	-----
F1	Credentials thật bị commit lên GitHub	P1 — bảo mật	2-4h (+ rotate key)
F2	Khởi động app mất ~41 giây	P1 — UX	4-6h
F3	<code>tab_bbox2.py</code> là bản fork trùng 99% <code>tab_bbox.py</code>	P1 — nợ kỹ thuật	1-3 ngày
F4	237 chỗ <code>except ...: pass</code> nuốt lỗi hoàn toàn	P1 — chẩn đoán	1-2 ngày (làm dần)
F5-F13	God class/function, thiếu test, settings ghi mỗi phím gõ, ...	P2	xem §3

0b. Trạng thái thực hiện (cập nhật 2026-07-26, sau khi user duyệt)

#	Vấn đề	Trạng thái	Ghi chú
---	-----	-----	-----
F1	Credentials trong git	🔒 Bỏ qua theo quyết định user	Key vẫn còn trong source + git history. Rủi ro được chấp nhận có ý thức
F2	Khởi động 41s	✅ Xong — còn 2,1s	Xem "Kết quả đo" bên dưới
F3	<code>tab_bbox2.py</code> trùng lặp	✅ Xong — đã xóa	Giữ <code>tab_bbox.py</code> (bản có đủ 40 hàm batch-add-class); <code>bbox2</code> chỉ hơn đúng 1 hàm <code>__len__</code>
F4	<code>except: pass</code> nuốt lỗi	⬢ Xong phần nguy	<code>_cfg_save</code> nay trả bool + báo user khi đóng app. 236 chỗ còn lại sửa dần

		hiếm nhất	
F5	God class / god function	☐ Chưa làm	Cần test coverage trước (F7)
F6	Settings ghi mỗi phím gõ	✓ Xong	Debounce 0,5s qua <code>_cfg_save_soon()</code> , flush khi đóng app
F7	Không có test	☐ Chưa làm	Có smoke test tạm ở <code>temp/audit-tool/smoke.py</code> , chưa thành bộ test chính thức
F8	Thiếu <code>requirements.txt</code>	✓ Xong	Kèm cảnh báo về torch+cu121 và 3 gói opencv trùng
F9	<code>from tkinter import *</code>	☐ Chưa làm	Sửa dẫn khi chạm file, không làm một lượt
F10	Settings mở cô + comment sai	✓ Xong	Sửa comment, xóa <code>.kztek_tools_settings.json</code> ở gốc
F11	<code>shell=True</code>	✓ Xong	Đổi sang <code>os.startfile</code> (kèm thêm <code>import os</code> vốn đang thiếu)
F12	<code>torch.load(weights_only=False)</code>	☐ Chưa làm	Ràng buộc thật của YOLOv5 checkpoint; chỉ nên thêm cảnh báo UI
F13	Vi phạm quy ước tài liệu	✓ Xong	<code>code-graph/CODE-GRAPH.md</code> + <code>.docx</code> ; <code>.gitignore</code> thêm <code>temp/</code> , <code>_workspace/</code>

Phát hiện bổ sung trong lúc sửa (không có trong audit gốc)

Thủ phạm khởi động chậm lớn nhất không phải `rfdetr` mà là `paddleocr`.

`tool/core/imports.py` — file được `app.py` import ở dòng 5, tức là thứ đầu tiên chạy —

có `from paddleocr import PaddleOCR` ở mức module. Dòng này kéo theo `torch` +

`transformers` + `matplotlib` ngay khi mở app, kể cả khi user không bao giờ mở tab OCR.

Audit tĩnh không thấy vì `paddleocr` không nằm trong danh sách thư viện nặng tôi quét;

chỉ lộ ra khi hook `builtins.__import__` để truy vết ai gọi `import torch`.

Bài học: quét regex theo tên thư viện đã biết là chưa đủ — phải truy vết import thật.

Kết quả đo

Giai đoạn	Trước	Sau
---	---	---
<code>import tool.core.app</code>	41,15s	1,06s
<code>App()</code> — dựng widget	3,93s	0,35s
Vẽ layout lần đầu	1,66s	~1,2s
Tổng khởi động	~46s	2,08s
Số tab dựng lúc khởi động	17/17	1/17
<code>torch</code> / <code>rfdetr</code> / <code>matplotlib</code> / <code>transformers</code> nạp lúc khởi động	có	không

Mở tab lần đầu tốn thêm 0,3–2,0s tùy tab (Train nặng nhất: 2,04s) — chi phí này đã được dời từ lúc khởi động sang đúng lúc user cần.

Kiểm thử đã chạy

temp/audit-tool/smoke.py — dựng app, mở lần lượt cả 17 tab, kiểm tra debounce settings:

```
KET QUA: 0 tab loi (17/17 tab dựng thành công)
flush ok: True | ghi dung gia tri: True | loi: ''
```

⚠ **Chưa kiểm thử:** thao tác thật bên trong từng tab (vẽ bbox, chạy train, OCR, detect).

Smoke test chỉ xác nhận tab dựng được và app không vỡ. Đường code đung vào việc nạp

model (`_load_model_any`, `_get_engine` của OCR) cần user chạy tay ít nhất 1 lần —

đây là chỗ rủi ro nhất của thay đổi lazy import.

1. Số liệu nền

Chỉ số	Giá trị
---	---
File .py (bỏ <code>__pycache__</code>)	89
Tổng số dòng	44.608
File > 1.000 dòng	9
File > 3.000 dòng	3
Tổng <code>except</code> handler	706
<code>bare except:</code>	0 ✓
<code>except ...: pass</code> (nuốt lỗi)	237 ⚠
Hàm > 150 dòng	37
Class > 800 dòng	12
<code>from tkinter import *</code>	45
<code>threading.Thread</code>	93 (89 có <code>daemon=True</code>)
<code>.after(...)</code> (marshal về UI thread)	284
File test tự động	0 ✗
<code>requirements.txt</code> / <code>pyproject.toml</code>	không có ✗
<code>compileall</code>	✓ exit 0
Mutable default arg	0 ✓
<code>os.system</code> / <code>verify=False</code>	0 ✓

2. Phát hiện P1

F1 — Credentials thật bị commit lên GitHub remote 🔒 BẢO MẬT

Bằng chứng:

- `tool/core/constants.py:41-63` — hardcoded sẵn trong source, đang được git track:
 - LottelImage: API base + username + password + MinIO endpoint + access key + secret key (IP công khai 119.17.223.230)
 - Parkingv6: API URL + MinIO endpoint + access key + secret key (IP công khai 113.162.247.111)
 - Parkingv8: client_id + client_secret
- Hai file settings đang được git track và có giá trị thật đã điền:

- `.kztek_tools_settings.json` (124 key) — `p8.cfg_user`, `p8.cfg_pass`, `p6.cfg_token` đều có giá trị
- `tool/.kztek_tools_settings.json` (223 key) — tương tự
- Remote: <https://github.com/flick-git-anhvn/TrainImageTool.git>

Tác động: access key MinIO + tài khoản API của hệ thống parking thật nằm trong lịch sử git đã push. Ai clone repo đều đọc được. Xóa file ở commit mới **không** đủ — giá trị vẫn còn trong history.

Đề xuất (theo thứ tự):

1. **Rotate ngay** toàn bộ key/password đã lộ (MinIO AK/SK của cả 2 cụm, password `kztek`, `p8_client_secret`, token `p6`). Đây là việc phải làm trước, không phụ thuộc việc sửa code.
 2. Chuyển giá trị mặc định trong `constants.py` sang đọc từ biến môi trường hoặc file `config.local.json` **không** track git; giữ lại trong source chỉ placeholder rỗng.
 3. Thêm vào `.gitignore`: `.kztek_tools_settings.json`, `**/.kztek_tools_settings.json`, rồi `git rm --cached` cả 2 file.
 4. Dọn history bằng `git filter-repo` (hoặc chấp nhận rủi ro nếu repo private và đã rotate xong).
- ⚠ Bước 4 viết lại history — cần user xác nhận trước khi chạy, và mọi máy đã clone phải re-clone.

F2 — App mất ~41 giây mới mở được 🌀 UX

Đo thực tế: `import tool.core.app` = 41,15 giây (máy hiện tại, Python 3.10.11).

Bóc tách chi phí import:

Thư viện	Thời gian	Import ở đâu (mức module, chạy ngay khi khởi động)
---	---	---
<code>rfdetr</code>	11,49s	<code>tool/features/detection/yolo_model_mixin.py:16</code>
<code>torch</code>	7,84s	<code>tool/features/detection/tab_slot_classifier.py:26</code>
<code>tab_lpr_tester</code>	4,44s	kéo theo phụ thuộc nặng
<code>matplotlib</code> (TkAgg)	0,21s	<code>tab_train.py:23-26</code> , <code>tab_classifier.py:25</code>
<code>ultralitics</code>	0,39s	<code>tab_classifier_tester.py:36</code> , <code>yolo_detect_all_mixin.py:24</code>

Nguyên nhân thứ hai — dựng tab háo hức: `tool/core/app.py:144-156` gọi `_wrap_scrollable()` cho cả 18 tab, rồi mới ẩn những tab user đã tắt trong "🔕 Cài đặt Tab". Nghĩa là tab bị tắt vẫn tốn đủ thời gian import + dựng widget.

Đề xuất:

1. Đưa `import torch / from rfdetr import ... / from ultralitics import ... / matplotlib` xuống **trong hàm** dùng tới chúng (lazy import). Riêng 2 dòng này đã cắt được ~19s.
2. Dựng tab lười: chỉ `_wrap_scrollable()` khi tab được `<<NotebookTabChanged>>` lần đầu; tab đang tắt thì không dựng.
3. Thêm splash screen ngắn để lần khởi động đầu (sau khi lazy hóa vẫn còn vài giây) không trông như treo.

Kỳ vọng sau khi sửa: khởi động còn ~2-4 giây, chi phí nạp model dời sang lúc user thực sự bấm Detect/Train.

F3 — `tab\_bbox2.py` trùng 99% với `tab\_bbox.py` ● NỢ KỸ THUẬT

Bảng chứng:

	`tab_bbox.py`	`tab_bbox2.py`
---	---	---
Số dòng	4.242	3.016
Số hàm	162	123
Class chính	BBoxEditorTab (4.721 dòng)	BBoxEditorTab2 (3.357 dòng)

122/123 hàm của `tab\_bbox2.py` trùng tên với `tab\_bbox.py` (99%) — `_apply_drag`, `_apply_filters`, `_auto_load_det_model`, `_autosave`, `_check_missing_labels`, `_compute_iou`, `_delete_current_image`, ...
Cả hai đều được đăng ký thành tab riêng trong `app.py:120-121` ("✂ BBox Editor" và "✂ BBox v2").

Tác động: mọi bug fix hoặc tính năng mới của BBox Editor phải sửa **hai lần**, ở hai file tổng 7.258 dòng. Đây là nguồn regression lớn nhất trong codebase hiện tại — sửa một bên quên bên kia là chuyện gần như chắc chắn xảy ra.

Đề xuất: không refactor mù. Làm theo thứ tự:

1. Diff cụ thể 2 file để chốt **v2 khác v1 đúng ở chỗ nào** (ngghi ngờ: layout `_build()` 592 dòng vs 538 dòng, và một số flow filter).
2. Nếu v2 là bản kế thừa v1 → **xóa v1**, đổi tên v2 thành BBox Editor duy nhất.
3. Nếu cả hai đều còn dùng → tách phần chung ra mixin (`bbox_core_mixin.py`), 2 tab chỉ giữ phần khác biệt.

Quyết định 1 hay 2 cần user/Tech Lead chốt — audit này không tự quyết.

F4 — 237 chỗ `except ...: pass` nuốt lỗi hoàn toàn

Phân bố:

File	Số chỗ
---	---
<code>features/training/tab_train.py</code>	29
<code>features/detection/tab_lpr_tester.py</code>	22
<code>features/annotation/tab_bbox.py</code>	21
<code>features/annotation/tab_bbox2.py</code>	16
<code>features/training/tab_classifier.py</code>	14
<code>features/detection/yolo_video_window_mixin.py</code>	12
<code>features/detection/yolo_detect_all_mixin.py</code>	10
còn lại (rải rác)	113

Trường hợp nguy hiểm cụ thể — `tool/core/settings.py:23-28`:

```
def _cfg_save():
    try:
        _SETTINGS_FILE.write_text(...)
    except Exception:
        pass          # ổ đĩa đầy / file read-only → user MẤT TOÀN BỘ cấu hình mà không
                     # hề biết
```

Đề xuất: không cần sửa hết 237 chỗ. Ưu tiên:

1. Các `except: pass` bao quanh I/O ghi file / lưu nhân / lưu cấu hình → phải log hoặc báo user.
2. Các chỗ bao quanh thao tác Tkinter cosmetic (bind, focus, geometry) → chấp nhận được, giữ nguyên.
3. Thêm 1 helper `_swallow(context: str)` ghi vào log app thay vì `pass` trần — đổi dần theo từng lần chạm file.

3. Phát hiện P2

F5 — God class / god function

File	Class	Dòng
---	---	---
features/training/tab_train.py	TrainTab	4.974
features/annotation/tab_bbox.py	BBoxEditorTab	4.721
features/annotation/tab_bbox2.py	BBoxEditorTab2	3.357
features/training/tab_classifier.py	ClassifierTrainTab	2.048
features/detection/tab_lpr_tester.py	LprTesterTab	1.720

Hàm dài nhất — 37 hàm vượt 150 dòng:

Dòng	Vị trí
---	---
789	yolo_video_window_mixin.py:35 _launch_video_window()
592	tab_bbox.py:202 _build()
545	yolo_eval_validate_mixin.py:34 _validate_true_folder()
538	tab_bbox2.py:169 _build()
468	tab_train.py:4556 _generate_html_report()
388	yolo_video_window_mixin.py:392 _worker()

Hướng tách rõ ràng nhất (rủi ro thấp, giá trị cao): các hàm `_build*`() khổng lồ → tách theo từng nhóm widget, giống cách `tab_yolo.py` đã được tách thành 17 mixin (7.017 → 195 dòng). Đó là tiền lệ tốt sẵn có trong chính repo này.

F6 — Settings ghi ra đĩa mỗi lần gõ 1 ký tự

`tool/core/settings.py:31-45` — `_bind_cfg()` gắn `var.trace_add("write", _cb)`, mà `_cb` gọi thẳng `_cfg_save()` → **serialize + ghi lại toàn bộ file JSON 223 key sau mỗi ký tự** user gõ vào bất kỳ Entry nào có bind `cfg`.

Đề xuất: debounce bằng `after(500, ...)` — hủy timer cũ mỗi lần write, chỉ ghi khi user ngừng gõ.

F7 — Không có test tự động nào

`pytest 9.0.3` đã cài nhưng repo có **0 file test**. Với 44.6k dòng và các module thuần logic dễ test (`shared/label_io.py`, `shared/cv_segment.py`, `shared/sam_utils.py`, `features/dataset/tab_split.py` phần chia tỉ lệ), đây là khoảng trống lớn — và cũng là điều kiện chặn theo CLAUDE.md \$WF-REFACTOR (yêu cầu coverage ≥ 80% trước khi refactor).

Đề xuất: bắt đầu từ `tool/shared/` — pure function, không đụng Tkinter, viết test rõ nhất và chặn regression cho phần F3/F5 sẽ refactor sau.

F8 — Không có `requirements.txt` / `pyproject.toml`

Dự án phụ thuộc `torch 2.5.1+cu121`, `ultralytics 8.4.42`, `opencv` (3 biến thể cùng lúc: `opencv-python`, `opencv-python-headless`, `opencv-contrib-python` — dễ xung đột), `rfdetr`, `onnxruntime`, `pillow-avif-plugin`... nhưng không khai báo ở đâu. Máy mới không dựng lại được môi trường; `ERRORS.md/CODE_GRAPH.md` đã ghi nhiều lỗi do lệch phiên bản CUDA/torch.

Đề xuất: `pip freeze` lọc lấy dependency trực tiếp → `requirements.txt`, ghi rõ index URL cho `torch+cu121`. Đồng thời gỡ bớt 2 trong 3 gói `opencv`.

F9 — `from tkinter import *` ở 45 file

Gây shadow tên (`Label`, `Menu`, `Entry`, `Text`, `Button`...) và làm IDE/linter mất khả năng phát hiện tên sai. Là nguyên nhân gián tiếp khiến không dùng được linter tĩnh trên repo này.

Đề xuất: đổi dần sang `import tkinter as tk` khi chạm file — không làm một lượt (45 file, rủi ro cao, lợi ích thấp nếu làm gấp).

F10 — File settings mờ côi ở thư mục gốc + comment sai

`tool/core/settings.py:9-10:`

```
# Chạy tu script: lưu config ở thư mục gốc dự án (parent của tool/)
_SETTINGS_FILE = Path(__file__).parent.parent / ".kztek_tools_settings.json"
```

`Path(__file__).parent = tool/core/` → `.parent.parent = `tool/``, không phải thư mục gốc dự án như comment nói (muốn ra gốc phải là `.parent.parent.parent`). Hệ quả: sau lần restructure sang `tool/core/`, file settings âm thầm đổi chỗ; `d:\Tool\.kztek_tools_settings.json` (124 key) hiện là **file chết** không ai đọc, còn file thật là `tool/.kztek_tools_settings.json` (223 key).

Đề xuất: chốt 1 vị trí duy nhất (khuyến nghị `%APPDATA%\KZTEK\image-tools\settings.json` — không nằm trong repo, tránh luôn F1), sửa comment cho khớp, xóa file mờ côi.

F11 — `shell=True` khi mở file bằng app mặc định

`tool/utils/bad_image_viewer.py:1331-1332:`

```
subprocess.Popen(["start", "", p], shell=True)
```

Tên file chứa `&`, `|`, `"` sẽ được cmd.exe diễn giải → chèn lệnh. Với tool nội bộ xử lý ảnh tải từ web/camera thì rủi ro thấp nhưng không bằng 0.

Đề xuất: thay bằng `os.startfile(p)` — đúng chức năng, không qua shell, ngắn hơn.

F12 — `torch.load(..., weights_only=False)`

`tool/features/detection/tab_slot_classifier.py:384` — load file `.pt` do user chọn với `weights_only=False`, tức là unpickle tùy ý → file model độc hại có thể thực thi code. Đây là ràng buộc thật của YOLOv5 raw checkpoint (không bỏ được dễ), nên xử lý bằng cảnh báo là đủ.

Đề xuất: thêm cảnh báo trong UI ("chỉ load model từ nguồn tin cậy") và ghi chú vào `ERRORS.md/GOTCHAS.md`, không đổi hành vi.

F13 — Vi phạm quy ước tài liệu của chính workspace

---	---
CLAUDE.md §17: bản đồ code phải ở <code>code-graph/CODE-GRAPH.md</code>	File thật ở <code>CODE_GRAPH.md</code> (thư mục gốc), thư mục <code>code-graph/</code> không tồn tại
CLAUDE.md §17.4: phải có <code>CODE-GRAPH.pdf</code> đồng bộ	Không có
CLAUDE.md §17.5: lạc hậu > 30 ngày phải viết lại	Sửa lần cuối 2026-07-05 , commit mới nhất 2026-07-24 → lệch 19 ngày, chưa tới ngưỡng nhưng đang trôi
Global CLAUDE.md: <code>.gitignore</code> phải có <code>temp/</code>	Thiếu (<code>temp/</code> , <code>_workspace/</code> đều chưa có)

Đề xuất: đổi `CODE_GRAPH.md` → `code-graph/CODE-GRAPH.md`, xuất PDF bằng `scripts/md_to_docx_kztek.py --no-docx`, thêm `temp/` + `_workspace/` vào `.gitignore`.

4. Những điểm đang làm tốt (giữ nguyên)

- **0 lỗi cú pháp** — `compileall` sạch 89/89 file.
- **0 `bare except:`** — mọi handler đều bắt kiểu cụ thể hoặc `Exception`, không nuốt `KeyboardInterrupt/SystemExit`.
- **0 mutable default argument, 0 `os.system`, 0 `verify=False`**.
- **Threading đúng mô hình Tkinter:** 93 thread thì 89 có `daemon=True`, và 284 lần `.after(...)` cho thấy kết quả được marshal về UI thread thay vì dựng widget từ thread nền — đây là chỗ dễ sai nhất trong app Tkinter và ở đây đã làm đúng.
- **Tách module theo domain rõ ràng:** `core/` (app, settings, constants, ui_helpers) – `features/{annotation, training, detection, dataset, collection, analysis}` – `shared/` (pure logic) – `utils/`.
- **Có tiền lệ refactor tốt:** `tab_yolo.py` đã được tách từ 7.017 → 195 dòng qua 17 mixin. Mẫu này áp dụng lại được cho F3/F5.
- Chỉ **2 dòng TODO/FIXME** trong toàn bộ 44.6k dòng.
- 2 kết quả `eval()` trong lần quét đầu là **đương tính giả** — đó là `model.eval()` của PyTorch, không phải `eval()` builtin.

5. Thứ tự đề xuất thực hiện

Bước	Việc	Ước tính	Workflow phù hợp
---	---	---	---
1	Rotate credentials đã lộ (F1 bước 1) — làm ngay, độc lập với code	1-2h	— (thao tác hạ tầng)
2	Gỡ secret khỏi source + gitignore settings (F1 bước 2-3)	2-4h	WF-HOTFIX
3	Lazy import <code>torch/rfdetr/ultralytics</code> (F2 mục 1)	2-3h	WF-FASTTRACK
4	Lazy dựng tab (F2 mục 2)	3-4h	WF-BUGFIX
5	Thêm <code>requirements.txt</code> (F8) + <code>.gitignore temp/</code> (F13)	1h	WF-FASTTRACK

6	Chốt hướng xử lý <code>tab_bbox2.py</code> (F3) — cần user quyết trước	0,5 ngày khảo sát	WF-REFACTOR bước 1-2
7	Test cho <code>tool/shared/</code> (F7) trước khi refactor F3/F5	2-3 ngày	WF-TEST
8	Debounce settings (F6), <code>os.startfile</code> (F11), dọn <code>except: pass</code> I/O (F4)	làm dần	WF-FASTTRACK
9	Tách god class (F5) — chỉ sau khi có test ở bước 7	1-2 tuần	WF-REFACTOR

Quyết định cần từ user trước khi đi tiếp:

- F1 bước 4: có viết lại git history không (ảnh hưởng mọi clone hiện có)?
- F3: `tab_bbox2` thay thế `tab_bbox`, hay giữ cả hai và tách mixin?

6. Phương pháp & giới hạn của audit này

Đã làm: phân tích tĩnh bằng AST trên 89 file (`temp/audit-tool/analyze.py`), quét pattern rủi ro theo regex (`temp/audit-tool/grep2.py`), đo thời gian import thực tế (`temp/audit-tool/imptime.py`), đối chiếu file được git track, đọc trực tiếp `core/app.py`, `core/settings.py`, `core/constants.py` và các vị trí được nêu.

Chưa làm (nằm ngoài phạm vi "audit, chưa sửa code"):

- Chưa chạy app thật để kiểm tra lỗi runtime / UI (cần `ux-ui-reviewer` hoặc `qa-engineer`).
- Chưa đo hiệu năng lúc chạy (load ảnh, inference, render canvas) — chỉ đo thời gian khởi động.
- Chưa kiểm tra tính đúng đắn nghiệp vụ của thuật toán annotation/segment.
- Chưa diff chi tiết `tab_bbox.py` vs `tab_bbox2.py` ở mức dòng (mới so ở mức tên hàm).

Artifact phụ trợ: `temp/audit-tool/` (script phân tích + output thô) — có thể xóa sau khi đọc xong báo cáo.